

PERSONAL AZURE INFRASTRUCTURE

Infrastructure as Code via Terraform

Hub & Spoke Network · NSGs · Private Endpoints · Remote State · Repeatable Cloud Environments

Project Title	Personal Azure Infrastructure: IaC via Terraform
Date	September 27, 2026
Analyst	Bryan Sykes
Platform	Microsoft Azure
IaC Tool	Terraform 1.16.4 (Windows) / 1.16.3 (Ubuntu)
Azure Provider	hashicorp/azurerm v3.117.1
Execution Environment	VS Code on Windows desktop + Azure CLI v2.90.0
Remote State	Azure Blob Storage: sykestfstate / tfstate container
GitHub	github.com/MrBSykes/personal-azure-infrastructure-terraform-IAC
Coverage	Business problem, architecture, IaC walkthrough, troubleshooting log, business alignment

Analyst: Bryan Sykes | Home Lab Portfolio Project

1. The Business Problem

In federal IT and enterprise cloud environments, infrastructure is rarely built once. Agencies deploy the same environment types repeatedly. Development, staging, production, disaster recovery across multiple projects, contracts, and security boundaries. When that infrastructure is built manually through a cloud portal, three critical problems emerge:

- **Inconsistency:** Two engineers clicking through the Azure portal to build "the same" environment will produce subtly different results. Different NSG rules, different naming conventions, different TLS settings. In a federal environment subject to FedRAMP or CMMC audit, inconsistency means compliance failures.
- **No audit trail:** Manual portal deployments leave no version-controlled record of what was built, when, or by whom. Infrastructure becomes undocumentable. Auditors cannot verify configuration history.
- **Speed and repeatability:** Rebuilding a complex network environment manually after an incident, a contract change, or a new project award takes days. The same environment defined in Terraform deploys in minutes, identically, every time.

Infrastructure as Code (IaC) solves all three problems simultaneously. Every resource is defined in version-controlled code. Every deployment is identical and repeatable. Every change is documented in Git history. The environment can be destroyed and recreated in minutes. A capability that directly supports disaster recovery, contract transitions, and security incident response.

This project demonstrates that capability using Terraform and Microsoft Azure. Building a secure, multi-environment hub and spoke network that can be deployed consistently across development and production without a single manual configuration step.

1.1 Real-World Applicability

Scenario	Without IaC	With Terraform IaC
New contract award — spin up cloud environment	3-5 days of manual portal work	terraform apply — minutes
Security incident — rebuild compromised environment	Days to recreate, risk of misconfiguration	terraform destroy + apply — identical rebuild
FedRAMP audit — prove infrastructure configuration	Screenshots and manual documentation	Git history is the audit trail
Scale from dev to prod	Rebuild everything manually with human error risk	Change tfvars — same code, different config
Onboard new engineer to infrastructure	Tribal knowledge, verbal walkthroughs	Read the .tf files — self-documenting

2. Architecture

2.1 Hub and Spoke Network Topology

The hub and spoke network pattern is the standard architecture for enterprise and federal Azure deployments. A central hub VNet hosts shared services (gateways, Bastion, DNS, firewalls), while spoke VNets host workloads (dev, prod, staging). All traffic between spokes must route through the hub enforcing centralized security inspection and control.

Component	Name	Address Space	Purpose
Hub VNet	hub-vnet	10.0.0.0/16	Shared services, gateway, Bastion (future)
Hub Gateway Subnet	GatewaySubnet	10.0.0.0/24	Reserved for VPN/Bastion gateway deployment
Dev Spoke VNet	dev-vnet	10.1.0.0/16	Development workloads — isolated from prod
Dev Private Subnet	private-subnet	10.1.1.0/24	VM, storage private endpoint, NSG enforced
Prod Spoke VNet	prod-vnet	10.2.0.0/16	Production workloads — isolated from dev
Prod Private Subnet	prod-subnet	10.2.1.0/24	Production resources, NSG enforced (future)

2.2 Security Design Principles

- **Default deny** — NSG rules deny all inbound traffic by default. Only explicitly allowed traffic is permitted.
- **Private by default** — Storage account blocks all public internet access. The private endpoint is the only access path.
- **Least privilege** — SSH inbound is only permitted from the Hub VNet address space (10.0.0.0/16), not from the internet.
- **Dev/prod isolation** — Dev and Prod spokes are peered only to the Hub, never to each other. A compromise in dev cannot laterally move to prod.
- **Encrypted transport** — TLS 1.2 minimum enforced on all storage accounts.

2.3 Remote State Architecture

Terraform state is stored in Azure Blob Storage rather than locally. This is the production-standard approach for several reasons: state is never lost if a local machine fails, multiple team members can collaborate on the same infrastructure, and state locking via the storage account prevents concurrent modifications from corrupting the state file.

Component	Value
Resource Group	TerraformStateRG
Storage Account	sykestfstate (Standard LRS, TLS 1.2)
Container	tfstate
State File Key	dev.terraform.tfstate

3. Terraform IaC Walkthrough

3.1 Provider and Backend Configuration

The terraform block defines the Azure provider version and configures the remote state backend. Every subsequent terraform apply writes state to sykestfstate automatically.

```

terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
  backend "azurerm" {
    resource_group_name = "TerraformStateRG"
    storage_account_name = "sykestfstate"
    container_name       = "tfstate"
    key                  = "dev.terraform.tfstate"
  }
}
provider "azurerm" {
  features {}
  subscription_id = "<subscription-id>"
}

```

3.2 Hub and Spoke VNets

Three VNets are defined with non-overlapping address spaces a hard requirement for VNet peering to function. Terraform references between resources (`azurerm_resource_group.hub.location`) replace hardcoded values, ensuring resources stay in sync automatically.

3.3 VNet Peering

Azure VNet peering requires explicit configuration in both directions hub-to-spoke AND spoke-to-hub. A common mistake is configuring only one direction, resulting in one-way connectivity. Terraform makes this requirement visible in code and enforces it on every deployment.

```

# Four peering resources for two bidirectional connections
azurerm_virtual_network_peering.hub_to_dev
azurerm_virtual_network_peering.dev_to_hub
azurerm_virtual_network_peering.hub_to_prod
azurerm_virtual_network_peering.prod_to_hub

```

3.4 Network Security Group — Least Privilege

The NSG enforces a default-deny security posture with explicit allows for required traffic only. Priority numbers determine rule evaluation order — lower numbers are evaluated first.

Rule	Priority	Direction	Source	Port	Action
Allow SSH inbound	100	Inbound	10.0.0.0/16 (Hub only)	22	Allow
Allow HTTPS outbound	200	Outbound	Any	443	Allow
Deny all inbound	4096	Inbound	Any	Any	Deny

3.5 Storage Account with Private Endpoint

This is the key distinction between this project and the Linux to Azure Phase 1 project. The Phase 1 storage account (sykeslogstorage) used public endpoint access secured by RBAC and TLS reachable from the internet. This storage account (sykesdevstore) uses a Private Endpoint it is only reachable from within the dev-vnet private subnet and is completely invisible to the public internet.

- network_rules default_action = "Deny", all public access blocked at the storage account level
- Private Endpoint creates a private NIC inside the dev private subnet with a private IP from the 10.1.1.0/24 range
- DNS resolution for sykesdevstore.blob.core.windows.net resolves to the private IP inside the VNet
- Any request from outside the VNet receives a network-level rejection, not an authentication failure

3.6 Terraform Workflow

Command	Purpose
terraform init	Downloads provider, configures remote backend, creates lock file
terraform plan	Shows exactly what will be created/changed/destroyed. No changes made
terraform apply	Builds the infrastructure. Requires manual yes confirmation
terraform state list	Shows all resources currently tracked in state
terraform import	Brings an existing Azure resource under Terraform management
terraform destroy	Tears down all managed infrastructure cleanly

4. Resources Deployed

17 Azure resources deployed and tracked in Terraform state across 3 resource groups:

Terraform Resource	Azure Resource	Status
azurerm_resource_group.hub	hub-rg (East US)	✓ Deployed
azurerm_resource_group.dev	dev-rg (East US 2)	✓ Deployed
azurerm_resource_group.prod	prod-rg (East US)	✓ Deployed
azurerm_virtual_network.hub	hub-vnet (10.0.0.0/16)	✓ Deployed
azurerm_virtual_network.dev	dev-vnet (10.1.0.0/16)	✓ Deployed
azurerm_virtual_network.prod	prod-vnet (10.2.0.0/16)	✓ Deployed
azurerm_subnet.hub_gateway	GatewaySubnet (10.0.0.0/24)	✓ Deployed
azurerm_subnet.dev_private	private-subnet (10.1.1.0/24)	✓ Deployed
azurerm_subnet.prod_private	private-subnet (10.2.1.0/24)	✓ Deployed
azurerm_virtual_network_peering.hub_to_dev	hub-vnet → dev-vnet peering	✓ Deployed
azurerm_virtual_network_peering.dev_to_hub	dev-vnet → hub-vnet peering	✓ Deployed

azurerm_virtual_network_peering.hub_to_prod	hub-vnet → prod-vnet peering	✓ Deployed
azurerm_virtual_network_peering.prod_to_hub	prod-vnet → hub-vnet peering	✓ Deployed
azurerm_network_security_group.dev_nsg	dev-nsg (least privilege rules)	✓ Deployed
azurerm_subnet_network_security_group_association	dev-nsg → private-subnet	✓ Deployed
azurerm_storage_account.dev	sykesdevstore (public access denied)	✓ Deployed
azurerm_private_endpoint.dev_storage	dev-storage-pe (blob private endpoint)	✓ Deployed

Note — Linux VM (dev-vm): Planned component not deployed due to Azure free tier VM capacity restrictions in both eastus and eastus2 during project execution. The Terraform code is written, tested to the plan stage, and commented out in main.tf pending capacity availability. This is an Azure platform limitation, not a code deficiency. The IaC is production-ready and will deploy successfully when capacity is available.

5. Troubleshooting Log

Every issue encountered during this project is documented below with root cause analysis and resolution. In a real infrastructure team, this log represents the institutional knowledge that prevents the same problems from recurring across projects and engineers.

Issue 1 — Azure CLI Not Found After Terraform Init

Field	Detail
Symptom	terraform init returned: unable to build authorizer — exec: "az": executable file not found in PATH
Root Cause	Azure CLI was installed on sykes-ubuntu but not on the Windows desktop where Terraform was running
Resolution	Installed Azure CLI v2.90.0 on Windows via the official MSI installer from aka.ms/installazurecliwindows . Restarted VS Code to pick up the new PATH entry.
Business Lesson	Every machine that runs Terraform needs the full dependency stack — Terraform binary, cloud provider CLI, and authentication. Document this as an onboarding prerequisite for any team.

Issue 2 — Resource Provider Registration Error

Field	Detail
Symptom	terraform plan failed: Encountered an error whilst ensuring Resource Providers are registered
Root Cause	Terraform automatically attempts to register Azure Resource Providers on first run. Free tier accounts lack permission for automatic registration.
Resolution	Manually registered required providers via Azure CLI: az provider register --namespace Microsoft.Network, Microsoft.Compute, Microsoft.Storage
Business Lesson	In enterprise environments, Resource Provider registration is a one-time subscription setup step handled by platform teams. Document this as a prerequisite for new Azure subscription onboarding.

Issue 3 — ED25519 SSH Key Not Supported by Azure VMs

Field	Detail
Symptom	terraform plan failed: the provided ssh-ed25519 SSH key is not supported. Only RSA SSH keys are supported by Azure.
Root Cause	The existing ED25519 key (used for sykes-ubuntu SSH access) is not supported by the Azure VM agent. Azure Linux VMs require RSA keys.
Resolution	Generated a dedicated RSA 4096-bit key pair for Azure VM access: <code>ssh-keygen -t rsa -b 4096 -C "terraform-azure-vm" -f \$HOME\.ssh\id_rsa_azure</code>
Business Lesson	Maintain separate SSH key pairs for different platforms and purposes. ED25519 for Linux-to-Linux SSH, RSA for Azure VM management. Document key inventory as part of access management policy.

Issue 4 — VM SKU Capacity Restrictions in East US and East US 2

Field	Detail
Symptom	terraform apply failed: SkuNotAvailable. The requested VM size Standard_B1s is currently not available in location eastus / eastus2.
Root Cause	Azure free tier VM capacity (B-series) is shared across all free accounts and subject to regional availability constraints. East US and East US 2 had no available B1s or B1ms capacity during project execution.
Resolution	VM resource commented out in main.tf pending capacity availability. Code is production-ready and validated to the terraform plan stage. Will deploy when capacity opens or using paid subscription.
Business Lesson	In production environments, use reserved instances or dedicated hosts to guarantee VM capacity. Capacity planning is a real operational concern this is not a free-tier-only problem. Document VM SKU requirements and reserved capacity as part of infrastructure planning.

Issue 5 — VNet Peering Conflict After Region Change

Field	Detail
Symptom	terraform apply failed: Provider produced inconsistent result after apply on <code>azurerm_virtual_network_peering.dev_to_hub</code>
Root Cause	Changing the dev resource group from eastus to eastus2 to resolve VM capacity issues caused the existing VNet peering state to become inconsistent the peering existed in Azure but was no longer tracked in Terraform state.
Resolution	Used terraform import to bring the existing peering back under Terraform management: <code>terraform import azurerm_virtual_network_peering.dev_to_hub <resource-id></code>
Business Lesson	terraform import is the correct tool for reconciling real Azure resources with Terraform state. Understanding this command is essential for any team inheriting existing infrastructure or recovering from partial deployment failures.

Issue 6 — Terraform Provider Binary Exceeded GitHub 100MB File Limit

Field	Detail
Symptom	git push rejected: File <code>.terraform/providers/.../terraform-provider-azurerm_v3.117.1_x5.exe</code> is 227.63 MB; exceeds GitHub file size limit of 100MB.

Root Cause	The .terraform directory (containing the downloaded provider binary) was included in the initial git commit before a .gitignore was in place.
Resolution	Created .gitignore excluding .terraform/, *.tfstate, *.tfstate.backup. Used git filter-branch to remove the binary from all git history, then force pushed to GitHub.
Business Lesson	Always create a .gitignore before the first git commit on any Terraform project. The .terraform directory should never be committed providers are downloaded by terraform init and should not be version controlled.

6. Business and Career Alignment

6.1 Federal IT and GovTech Relevance

Federal IT contracting firms, including CACI, SAIC, HII, and Leidos increasingly require cloud engineers who can build and maintain infrastructure through code rather than manual portal operations. Several federal frameworks directly mandate or strongly prefer IaC practices:

- **FedRAMP** — Requires documented, reproducible infrastructure configurations. Terraform code IS the documentation.
- **CMMC Level 2/3** — Configuration management controls require that system configurations be controlled, versioned, and auditable. Git-tracked Terraform meets this requirement.
- **NIST 800-53 CM-2** (Baseline Configuration) — Requires organizations to maintain baseline configurations. A Terraform state file and Git history constitute a verifiable baseline.
- **Zero Trust Architecture** — The private endpoint and default-deny NSG design in this project directly implements Zero Trust network principles at the infrastructure layer.

6.2 Skills Demonstrated

Skill	Evidence in This Project
Infrastructure as Code (Terraform)	17 Azure resources defined, deployed, and managed entirely through .tf files
Azure Networking	Hub and spoke VNet topology, VNet peering (bidirectional), subnet design, NSG rules
Cloud Security	Default-deny NSGs, private endpoints, TLS enforcement, no public VM IPs, dev/prod isolation
Remote State Management	Azure Blob Storage backend configured with state locking via storage account
Version Control	Git repository with proper .gitignore, meaningful commit messages, GitHub published
Troubleshooting	6 real issues diagnosed and resolved provider errors, SSH key compatibility, capacity constraints, state reconciliation
Documentation	Full PDF documentation, README, troubleshooting log with business context. Everything reviewable by a hiring manager

6.3 Certification Alignment

Certification	Relevance to This Project
---------------	---------------------------

AZ-900 ☒	Azure resource groups, VNets, storage accounts, subscriptions, security concepts — all demonstrated hands-on
AZ-104 (Next)	Virtual networking (25-30% of exam), NSGs, VNet peering, private endpoints, storage management — this project covers most of it
CompTIA Security+ ☒	Network segmentation, least privilege, defense in depth, Zero Trust — all implemented at the infrastructure layer

7. Next Steps

Action	Purpose
Deploy Linux VM when capacity available	Complete component 3, bare-metal VM in private subnet with no public IP
Add Azure Bastion to hub-vnet	Enable secure RDP/SSH to private VMs without public Ips, enterprise standard
Add prod NSG and storage account	Mirror dev security controls in prod spoke demonstrate environment parity via IaC
Refactor into Terraform modules	Extract reusable networking, NSG, and storage patterns into modules. Production-grade code structure
Add tfvars for dev/prod separation	Demonstrate how the same code deploys different configurations across environments
Pursue AZ-104	This project covers most of the AZ-104 virtual networking domain certification is the logical next step

8. Conclusion

This project demonstrates that a complete, secure Azure cloud environment. Hub and spoke networking, least-privilege NSGs, private storage endpoints, and remote state management that can be defined entirely in code and deployed in minutes without a single manual portal interaction. That is the core promise of Infrastructure as Code, and this project delivers it.

The troubleshooting challenges encountered along the way. Provider registration permissions, SSH key compatibility, VM capacity constraints, state reconciliation after region changes, and Git history rewriting are not edge cases. They are the real operational problems that infrastructure engineers face on actual federal contracts. Diagnosing and resolving them is the job.

The environment is live in Azure, the code is version-controlled on GitHub, and the entire deployment is reproducible with a single terraform apply. In a federal contracting context, that means a new contract award that once took days of manual infrastructure work can now be completed in minutes. Identically, auditably, and at scale.

Documentation prepared by: Bryan Sykes | September 27, 2026 | Personal Azure Infrastructure — IaC via Terraform